Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #6

The Brain: Flight-Phase State Machine as an RTOS Control Task

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: July 23, 2026

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Build a state machine in SCADE:** model an explicit automaton with states, an initial state, and guarded transitions in the SCADE One state-machine editor, instead of encoding state by hand with `pre` and counters.
- ✓ **Read how state is compiled:** identify the automaton's current state in the generated code as a dedicated `SSM_ST` memory field of the context — the same construct seen in advanced SCADE models.
- ✓ **Enforce legal transitions:** design the flight-phase machine so only lawful phase changes are possible, and justify why a state machine guarantees this where an `if/else` on an integer does not.
- ✓ **Run the machine as the control task:** drop the generated state machine into the Practice 5 RTOS skeleton, feed it events from a new input task, and map its abstract phase output to LED patterns in the glue.

2 Introduction

Every flight controller must know *what phase it is in* — on ground, taking off, cruising, or landing — because the correct behaviour of the control surfaces depends entirely on that phase. Encoding this with scattered flags and `if` statements is fragile: nothing stops an illegal jump from ground straight to landing. A *state machine* makes the phases and the legal transitions between them explicit and verifiable, which is exactly why safety standards favour them. This practice builds the flight-phase machine in the SCADE automaton editor, generates it, and runs it as the control task of the RTOS skeleton from Practice 5 — driven now by a real input event (the on-board button). It is the "mode" brain that the sensor, servo, and safety layers of the coming practices will obey.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (Ansys)	1	Automaton editor + code generation
2	ESP32 DevKit V1	1	Environment from Practice 2/3
3	USB-A to Micro-USB cable	1	Data cable (console)
4	ESP-IDF (v5.x)	1	FreeRTOS built in
5	Three LEDs + 220 Ω resistors	3	Phase indicator (GPIO 25/26/27)
6	On-board `BOOT` button	1	GPIO 0, active-low — no wiring

No new hardware

The only input is the DevKit's on-board `BOOT` button (GPIO 0, active-low), so nothing is wired beyond the three phase LEDs from earlier practices. Unlike Practices 3–5, this practice generates a *new* model — the flight-phase machine — so a fresh `scade_gen` component is produced; the traffic-light component is not reused.

4 Theory & Block Reference

This is the first practice to use the SCADE *automaton* (state machine) editor. A state machine is still a synchronous operator — it is evaluated once per cycle — but instead of dataflow equations it is drawn as states and transitions, and it remembers which state it is in between cycles. The reference below covers the three ideas needed to build, generate, and integrate one.

4.1. Why a state machine, not an integer and if/else

One could track the phase with a plain `int` and change it with `if` statements, exactly as the traffic-light counter did. The problem is that nothing constrains the changes: a bug could set the phase to any value, including an impossible jump. An automaton makes the *transitions* first-class: a state can only be left along a drawn arrow whose guard is true. Illegal transitions are not "unlikely" — they are *unrepresentable*. For a phase that decides how flight controls move, that guarantee is the whole point.

Concept: `Automaton (states + initial state)`

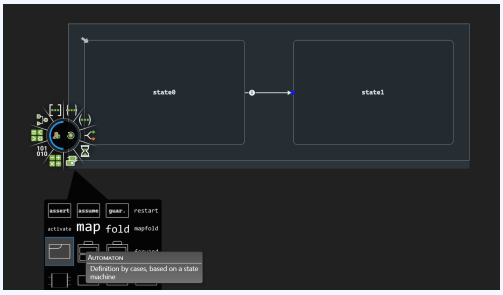

Figure 1: The four-state flight-phase automaton with its initial state marked — annotated view.

Key elements:

- **States:** each drawn box is a named state (`GROUND`, `TAKEOFF`, `CRUISE`, `LANDING`). Exactly one is active per cycle.
- **Initial state:** one state is marked initial — the machine starts there after `reset`. Generation fails if none (or more than one) is marked.
- **State equations:** inside each state you write the outputs it produces (here `phase = 0..3`).

Behavior:

The active state is remembered between cycles. On each cycle the machine emits the active state's outputs and then evaluates its outgoing transitions to decide the state for the next cycle.

Concept: Transition (guarded arrow)

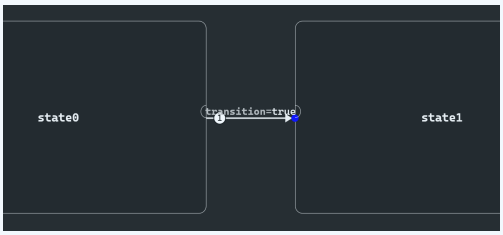

Figure 2: A transition arrow carrying the guard `next` between two states — annotated view.

Key elements:

- **Guard:** a Boolean condition on the arrow (here the input `next`). The transition is taken only in a cycle where the guard is true.
- **Direction:** arrows are one-way; the legal path is `GROUND`→`TAKEOFF`→`CRUISE`→`LANDING`→`GROUND`.
- **Determinism:** if two guards out of one state could be true at once, SCADE requires a priority so the choice is never ambiguous.

Behavior:
A guard of `true` (a constant) fires every cycle, so the machine would race through all states in four cycles — a classic mistake. Guard on the event, not on a constant.

Concept: Generated state: the SSM_ST memory

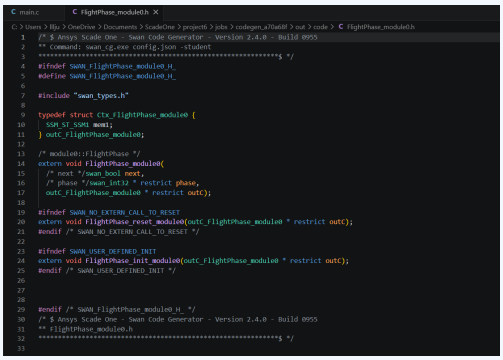

Figure 3: The generated context carrying the automaton's state as an `SSM_ST` field — annotated view.

Key elements:

- **`SSM_ST_...` field:** the code generator adds the current state to the context struct as an enum-typed memory (e.g. `mem_state`) — the same construct that appears in full aircraft models.
- **`reset`:** sets that field to the initial state.
- **`step`:** reads the input, emits the active state's outputs, and writes the next state back into the field.

Behavior:
The state machine compiles to exactly the same `context/reset/step` contract as every earlier model — the state is just one more remembered value. Nothing about the RTOS integration changes.

4.2. The phase-to-LED mapping (in the glue, not the model)

The model outputs an abstract `phase` (0–3); the glue decides how to show it. Four phases are distinguished on three LEDs as follows:

phase	state	green	yellow	red
0	GROUND	✓		
1	TAKEOFF	✓	✓	
2	CRUISE		✓	
3	LANDING		✓	✓

5 Worked Example

5.1. Objective

Build the smallest possible automaton — a two-state latch — to learn the editor and see the `SSM_ST` memory before the four-state machine. The button toggles one LED on and off. About 25 minutes.

5.2. Step-by-Step

1. **Create the operator and insert a state machine.**
   In a new SCADE One project, create an operator `Latch` with a `bool` input `btn` and a `bool` output `on`. Insert a *State Machine* into its body.

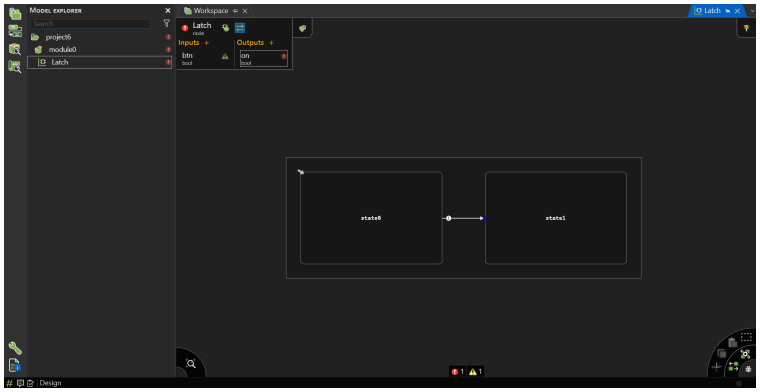

Figure 4: Empty state machine inserted into the `Latch` operator.

2. **Draw two states and the transitions.**
   Create states `OFF` and `ON`; mark `OFF` initial. In `OFF` write `on = false`; in `ON` write `on = true`. Draw a transition `OFF`→`ON` guarded by `btn`, and `ON`→`OFF` guarded by `btn`.

3. **Simulate.**
   In the simulator, toggle `btn` true for one cycle and confirm the active state flips and `on` follows. Note that when `btn` stays false the state is held — that held value is the `SSM_ST` memory.

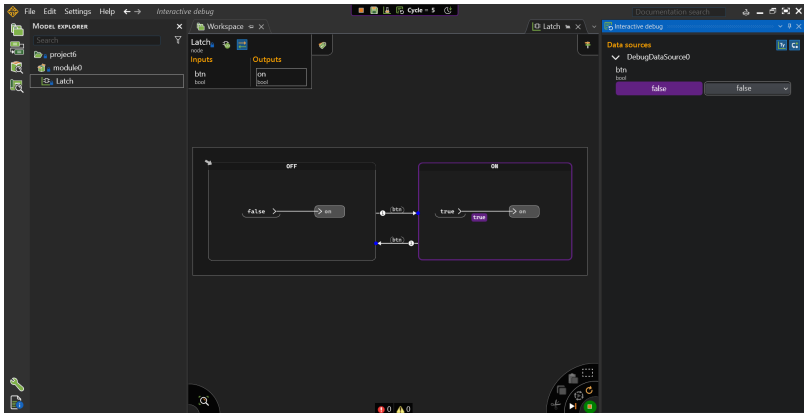

Figure 5: Simulator: a single `btn` pulse flips the active state; the state persists otherwise.

4. **Generate and read the header.**
   Run a Code Generation job (Practice 3 workflow). Open the generated header and find the `SSM_ST` field in the context struct — this is where the automaton's state lives. Confirm the step and reset function names for use in the glue.

6 Main Activity

6.1. Description

Build the four-state flight-phase machine `GROUND`→`TAKEOFF`→`xx`→`LANDING`→`GROUND`, each transition guarded by a one-cycle `next` event. Generate it and run it as the control task of the Practice 5 skeleton: an `input_task` debounces the `BOOT` button and posts a `next` event to a queue; the `model_task` steps the machine and sends the resulting `phase` to the LED task, which lights the pattern of Section 4. Pressing the button walks the aircraft through its flight phases.

6.2. Functional Specifications

- **Spec 1:** The SCADE model is an automaton with four states, exactly one initial state (`GROUND`), and transitions guarded by the input `next`; it outputs `phase` = 0–3.
- **Spec 2:** The sequence is validated in the SCADE simulator before generation: `next` advances one phase and wraps `LANDING`→`GROUND`; with `next` false the phase holds.
- **Spec 3:** An `input_task` (priority 2) detects a `BOOT` press as a falling edge, debounces it, and sends one event per press to `q_evt`.
- **Spec 4:** The `model_task` (priority 3) steps the machine every 100 ms, consuming at most one pending event per cycle as the `next` pulse, and sends `phase` to `q_led`.
- **Spec 5:** The `led_task` maps `phase` to the three-LED pattern of Section 4 and logs each phase change; no illegal phase is ever displayed.

6.3. Activity Steps

1. **Build the four-state machine.**
   Create operator `FlightPhase` with input `next` (`bool`) and output `phase` (`int32`). Draw the four states, mark `GROUND` initial, write `phase = 0..3` in each, and draw the four `next`-guarded transitions forming the ring.

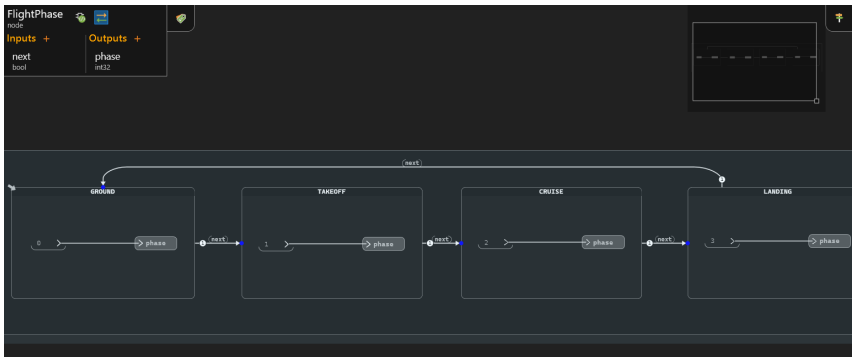

Figure 6: The `FlightPhase` automaton: four states, one initial, ring transitions guarded by `next`.

2. **Simulate and validate against Spec 2.**
   Pulse `next` and confirm the phase advances 0→1→2→3 →0 and holds when `next` is false. Then run a Code Generation job and note the new operator/context names.

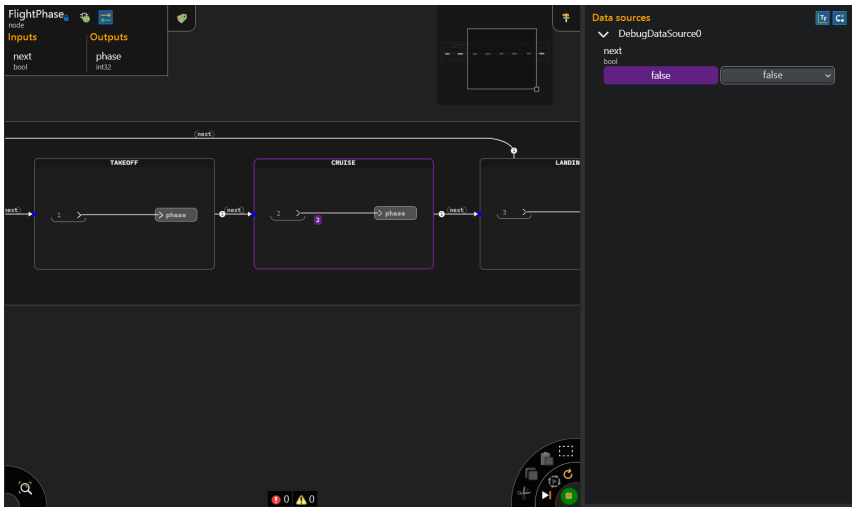

Figure 7: Simulator confirming the guarded phase sequence and the held state.

3. **Create the project and integrate the component.**
   Copy a working project as the base, drop in the freshly generated `scade_gen` (its `.c/.h`, the support headers, and the `swan_config.h` reused unchanged), and register every generated `.c` in the component `CMakeLists.txt` — the Practice 3 integration workflow, with the new operator names.

4. **Write the multi-task glue.**
   Replace `main.c`. Confirm the generated step signature in your header first (it will be `operatorX_module0(next, &phase, &ctx)` with context `outC_operatorX_module0`); adapt the names below.

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "operator0_module0.h"   // generated (FlightPhase) - confirm name

#define BTN     0            // BOOT button, active-low
#define LED_GRN 25
#define LED_YEL 26
#define LED_RED 27
static const char *TAG = "RTOS";

static QueueHandle_t q_evt;   // button events (uint8_t)
static QueueHandle_t q_led;   // phase (int)

// ---- input task: debounced BOOT button -> one event per press ----
static void input_task(void *arg)
{
    gpio_config_t io = { .pin_bit_mask = 1ULL << BTN,
        .mode = GPIO_MODE_INPUT, .pull_up_en = GPIO_PULLUP_ENABLE };
    gpio_config(&io);
    int prev = 1;
    while (1) {
        int now = gpio_get_level(BTN);
        if (prev == 1 && now == 0) {         // falling edge = press
            uint8_t ev = 1;
            xQueueSend(q_evt, &ev, 0);
        }
        prev = now;
        vTaskDelay(pdMS_TO_TICKS(20));        // poll + debounce
    }
}
```

```
// ---- control task: the state machine ----
static void model_task(void *arg)
{
    outC_operator0_module0 ctx;
    swan_int32 phase;
    operator0_reset_module0(&ctx);
    TickType_t last = xTaskGetTickCount();
    while (1) {
        uint8_t ev;
        swan_bool next = false;
        if (xQueueReceive(q_evt, &ev, 0) == pdTRUE) next = true; // 1 pulse/cycle
        operator0_module0(next, &phase, &ctx);
        int p = (int)phase;
        xQueueSend(q_led, &p, 0);
        vTaskDelayUntil(&last, pdMS_TO_TICKS(100));
    }
}

// ---- consumer: phase -> LED pattern (Section 4) ----
static void led_task(void *arg)
{
    gpio_set_direction(LED_GRN, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_YEL, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_RED, GPIO_MODE_OUTPUT);
    int p, last_p = -1;
    while (1) if (xQueueReceive(q_led, &p, portMAX_DELAY) == pdTRUE) {
        gpio_set_level(LED_GRN, (p == 0 || p == 1));
        gpio_set_level(LED_YEL, (p == 1 || p == 2 || p == 3));
        gpio_set_level(LED_RED, (p == 3));
        if (p != last_p) {                    // log only on change
            const char *n[] = {"GROUND","TAKEOFF","CRUISE","LANDING"};
            ESP_LOGI(TAG, "phase=%d %s", p, n[p]);
            last_p = p;
        }
    }
}

void app_main(void)
{
    q_evt = xQueueCreate(4, sizeof(uint8_t));
    q_led = xQueueCreate(4, sizeof(int));
    xTaskCreate(input_task, "input", 2048, NULL, 2, NULL);
    xTaskCreate(model_task, "model", 3072, NULL, 3, NULL);
    xTaskCreate(led_task,   "led",   3072, NULL, 1, NULL);
}
```

5. **Set the dependencies and flash.**
   `main\CMakeLists.txt` needs the generated component and GPIO:

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS "."
                       REQUIRES scade_gen esp_driver_gpio)
```

   Then `idf.py -p COM8 flash monitor`. Each `BOOT` press advances the phase by exactly one; the LEDs show the Section 4 pattern and the console logs the named phase.

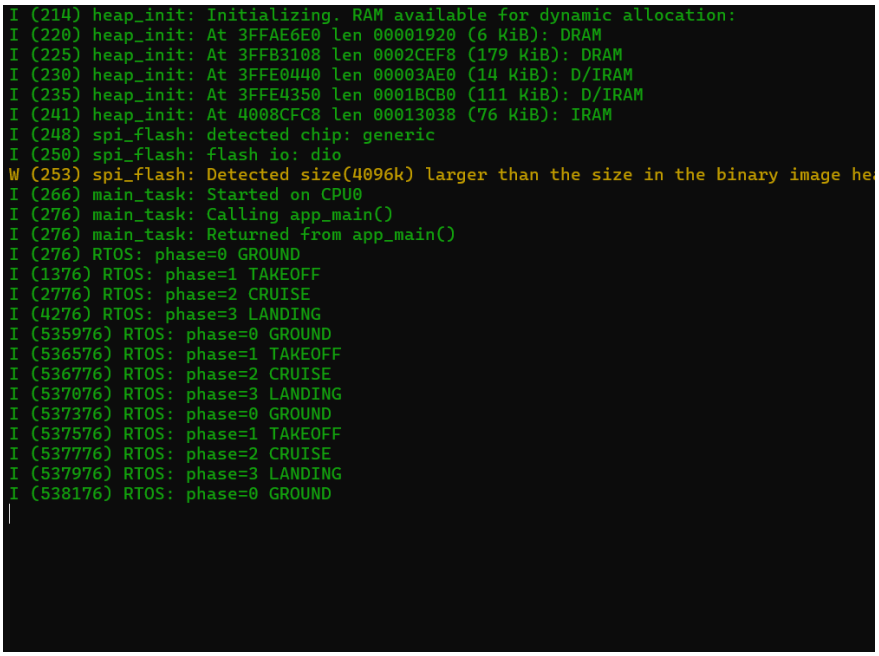

Figure 8: Console: each button press advances the flight phase by one, wrapping at `LANDING`.

7 Expected Results

Expected Results

- **SCADE Simulation:** Each `next` pulse advances `phase` by one and wraps `LANDING`→`GROUND`; with `next` false the phase is held. No sequence other than the drawn ring is reachable.
- **Console / UART Output:** One line per phase change (button press), naming the phase:

```
I (1204) RTOS: phase=1 TAKEOFF
I (2560) RTOS: phase=2 CRUISE
I (4010) RTOS: phase=3 LANDING
I (5361) RTOS: phase=0 GROUND
```

- **Physical Behavior:** Pressing `BOOT` steps the LEDs through the four patterns (green → green+yellow → yellow → yellow+red → green). One press = one phase; holding or releasing the button never skips a phase.

8 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [*Comprehension*] Open your generated header and locate the `SSM_ST` field in the context struct. Explain what it stores, what the `reset` function writes into it, and how the `step` function uses and updates it each cycle. Quote the relevant lines.

Q2. [*Analysis*] Your `model_task` consumes at most one event per 100 ms cycle. Predict what happens if the user presses the button twice within one cycle, and what happens to a press that arrives while `q_evt` already holds an event. Explain the role of the queue length here.

Q3. [*Analysis*] Compare your automaton with an equivalent implementation that stores the phase in a plain `int` and advances it with `phase = (phase+1) % 4` inside an `if`. Which illegal behaviours does the automaton make unrepresentable, and what could a bug do to the integer version that it cannot do to the automaton?

Q4. [*Synthesis*] Practice 8 will add an emergency mode reachable from *any* phase. Describe how you would extend this automaton: the new `EMERGENCY` state, the guard that enters it from every other state, and how the machine should return to `GROUND`. State what changes in the generated context and what changes in the glue.

Q5. [*Critical*] Suppose the transition guards were accidentally set to the constant `true` instead of `next`. Describe exactly what the LEDs and console would show, why it happens in terms of the per-cycle evaluation of the automaton, and how the simulator would have caught it before flashing.

9 Conclusion

Write a concise conclusion reflecting on what you implemented, what you observed, and what challenges you encountered. Connect your findings to model-based design for critical systems — in particular, what it means that legal flight-phase transitions are guaranteed

by the model's structure rather than by careful coding, and how this "brain" will govern the sensors, servos, and safety logic added in the practices ahead.

10 Bonus Challenge

★ Bonus Challenge

Challenge:
Make `TAKEOFF`→`CRUISE` automatic and timed instead of button-driven: once in `TAKEOFF`, the machine should advance to `CRUISE` on its own after a fixed number of cycles (say 30 = 3 s), while all other transitions stay on `next`. Add the counter *inside* the SCADE model (a `pre`-based tick counter within the `TAKEOFF` state), not in the glue. Re-simulate, regenerate, and confirm the aircraft climbs to cruise by itself.

Hint:
A state can contain its own dataflow. Inside `TAKEOFF`, count the cycles the state has been active and make the outgoing guard `next or count >= 30`. The counter resets naturally each time the state is re-entered.

11 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: Code generation fails: “no initial state” / “ambiguous transitions”

Symptom: The Code Generation job reports an error about a missing initial state or non-deterministic transitions.

Cause: An automaton must have exactly one initial state, and no two guards out of the same state may be simultaneously true without a declared priority.

Fix: Mark exactly one state initial (`GROUND`); ensure each state’s outgoing guard is the single event `next`. The simulator flags both problems before generation.

Issue: The phase races through all four states instantly

Symptom: On boot the LEDs flick straight to the last pattern; the console shows all four phases in one burst.

Cause: A transition guard was left as the constant `true` instead of `next`, so every transition fires every cycle.

Fix: Guard each transition on the `next` input, not on a constant. Re-simulate: with `next` false the phase must hold.

Issue: One button press advances several phases

Symptom: A single `BOOT` press jumps two or three phases at once.

Cause: Button bounce — the mechanical contact produces several edges, so the input task queues several events.

Fix: The 20 ms poll in `input_task` debounces most bounce; if it persists, raise the poll interval or require the level to be stable for two reads before sending the event.

Issue: The button never advances the phase (or advances constantly)

Symptom: Pressing `BOOT` does nothing, or the phase advances without any press.

Cause: `BOOT` is active-low and needs a pull-up: a press reads `0`, release reads `1`. A missing pull-up or an inverted edge test breaks the detection.

Fix: Configure GPIO 0 as input with `pull_up_en` enabled and detect the `1→0` (falling) edge, as in the listing.